

Why the trading bot lost money — and the engine built so it learns from it

A review of every logged trade found the strategy could not have made money — not through bad luck, but because its profit target was unreachable by construction. This document explains what went wrong, what was rebuilt, and what remains outstanding.

Period reviewed: 5–10 August 2026 Source: v2_trade_ledger (26 entries)
Status: deployed & verified

CONTENTS

- 1. Executive summary
- 2. Background — what the system does
- 3. The finding: a target that could never be reached
- 4. Why it kept losing — three compounding causes
- 5. The reporting problem: two different loss figures
- 6. What we built: the Evolution Engine
- 7. Defects found and fixed
- 8. Infrastructure and access
- 9. Outstanding items
- 10. What success realistically looks like
- 11. Glossary

1. Executive summary

The automated trading bot was configured with a **take-profit of 6%** — meaning it would only bank a winning trade once price moved 6% in its favour. During the entire five-day review period, Bitcoin's total price range was **2.24%**. The target was therefore impossible to reach. The **stop-loss of 2%**, however, was well within normal price movement.

The consequence is arithmetic, not opinion: across 11 closed positions the take-profit triggered **zero times**. Every position ended either at a stop-loss or by being forced out when the strategy's signal flipped — almost always at a loss. The system was configured so that only losing outcomes were available to it.

TAKE-PROFITS HIT	MARKET RANGE	WIN RATE	ACTUAL RESULT
0	2.24%	9%	-\$4,039
of 11 closed positions	against a 6% target	25% needed to break even	app displayed -\$1,388

Three things were done in response. First, the accounting defects that hid the problem were fixed — the trade counter, the reset function and the report page were all reporting incorrect figures. Second, an **Evolution Engine** was built: it studies closed trades, diagnoses this class of failure automatically, and

proposes corrections for human approval. Third, a series of reliability and interface defects were repaired, including bots vanishing on restart and the platform being unusable on mobile.

2. Background — what the system does

The platform runs automated trading strategies against live market prices using **paper trading**: simulated money, real prices. Each running *bot* pairs one strategy (for example Ichimoku, a trend-following method) with one instrument (for example BTCUSDT) and a set of risk parameters — how much to trade, when to take profit, and when to cut a loss.

Every executed trade is written to a permanent record called the **trade ledger**. This ledger is the authoritative history of what the system did, and it is the basis for every figure in this document.

3. The finding: a target that could never be reached

Each bot was configured with a take-profit of 6% and a stop-loss of 2%. In isolation this looks sensible — it is a 1:3 risk-to-reward ratio, meaning each win is worth three times each loss. On paper, such a configuration only needs to be right about **25% of the time** to break even.

The problem is that those percentages must be achievable by the instrument being traded. Over the review period Bitcoin traded between \$63,945 and \$65,378 — a total span of 2.24%. A position needing a 6% move to close profitably could never get there. Meanwhile a 2% adverse move was entirely ordinary.

ROOT CAUSE

The profit target was roughly three times larger than the entire price range of the market during the period, while the loss limit sat comfortably inside normal movement.

This produces a one-sided system: losses can trigger, gains cannot. No amount of strategy skill compensates for that, because the winning exit is simply unreachable.

How the 11 positions actually ended

EXIT REASON	COUNT	AVERAGE	BEST	WORST
Reversal / forced close	10	-\$258.18	+\$23.98	-\$538.64
Stop-loss	1	-\$1,457.83	-\$1,457.83	-\$1,457.83
Take-profit	0	—	—	—

Only one of eleven positions finished ahead, by \$23.98. The single stop-loss that did fire cost \$1,457.83 — more than sixty times the size of the only winner.

4. Why it kept losing — three compounding causes

Targets did not match the instrument

The bots traded on one-minute candles, where a typical bar moves around 0.03%. Reaching a 6% target would require roughly two hundred consecutive bars moving in the same direction without triggering the

much closer 2% stop first. This is not a realistic sequence.

A trend strategy running in a flat market

Ichimoku is a trend-following method: it profits when price moves persistently in one direction. The market during this period was range-bound — drifting sideways. In those conditions a trend strategy repeatedly signals a direction, gets contradicted, and reverses. This is known as being **whipsawed**, and it produced nine reversal exits, each locking in a modest loss.

Commission accumulated through over-trading

Total commission was **\$523.83** across 26 trades. Each reversal pays commission twice — once to close the existing position and once to open the opposite one. Roughly 13% of the total loss was spent on transaction fees generated by churn rather than by market movement.

5. The reporting problem: two different loss figures

The dashboard displayed a loss of \$1,388.15. The trade ledger totalled \$4,039.59. Both were produced by the same system.

The cause is that the simulated account rebuilds its cash balance from a fixed \$100,000 starting point every time the application restarts, discarding accumulated profit and loss in the process. Because the application was redeployed several times during the review period, the on-screen equity figure only ever reflected losses since the most recent restart.

PRACTICAL GUIDANCE

Use the trade ledger as the source of truth, not the equity figure on the dashboard. The ledger is written once per trade and persists permanently. The equity display will understate cumulative losses until account persistence is addressed — see Outstanding items.

6. What we built: the Evolution Engine

The underlying failure was not that a bad number was chosen once. It was that **nothing in the system noticed the number was wrong**, even after eleven consecutive trades demonstrated it. The Evolution Engine addresses that gap.

It examines every *closed* trade — closed, because only a finished trade has a known outcome — measures how positions are ending, diagnoses the problem, and proposes specific parameter changes. It does not act on its own: a person reviews the reasoning and approves it.

How a cycle works

- 1 Observe**
Collects completed trades for one strategy on one instrument. Open positions are ignored — their outcome is not yet known.
- 2 Diagnose**
Calculates win rate, average profit per trade, and the distribution of exit reasons — how often trades ended at target, at stop, or by reversal.
- 3 Propose**
Suggests bounded parameter adjustments, each accompanied by a written explanation of the evidence behind it.
- 4 Review**
A progress meter fills as evidence accumulates. When full, the proposed lesson appears on the home page for a person to read.
- 5 Apply**
On approval, the new parameters are pushed to running bots immediately and recorded against the current market conditions for future reference.

Tested against the real trade history

When run against the actual ledger, the engine independently reached the same conclusions that were reached by manual analysis — without being told what to look for:

```
take_profit  6.0% → 3.6%
stop_loss    2.0% → 1.6%
cooldown     0 bars → 5 bars
```

Its own stated reasoning, quoted directly:

"Take-profit never hit in 19 closed trades — target unreachable for this instrument's volatility. Lowering it so winners can actually be banked."

"95% of exits came from the signal flipping, and expectancy is negative — the strategy is being whipsawed in a ranging market. Demanding stronger conviction before entering."

"Average loss (\$450.03) is more than 2.5× the average win (\$23.98). Tightening the stop to cut the tail off losing trades."

Equally important, it **declined to change** the two strategies that had insufficient history, reporting that it was still gathering evidence. Restraint on thin data is the single most valuable behaviour in a system of this kind.

The five safeguards, and why each exists

SAFEGUARD	PURPOSE
Interpretable	Every proposed change carries a plain-language reason. The question “why did it do that?” always has an answer, which makes the system auditable rather than a black box.
Evidence-gated	No changes are proposed below 8 completed trades, and re-evaluation happens every 5 thereafter. Adjusting parameters on a handful of trades fits random noise rather than real behaviour.
Bounded	Each parameter has a hard minimum and maximum, and each adjustment is a small step. This prevents a runaway sequence of changes drifting into nonsensical settings.
Reversible	The best-performing configuration seen so far is retained. If a change makes results worse, the engine proposes reverting to it.
Fail-safe	A strategy that remains unprofitable across three generations is proposed for pausing, so it stops losing money while others continue.

What it remembers, and where

Learning is stored separately for each **strategy and instrument combination**. Ichimoku on Bitcoin and Ichimoku on Ethereum keep entirely separate lessons, because their price behaviour differs by an order of magnitude — a stop that is sensible on one would be far too tight or too loose on the other. The same mechanism applies to stocks, foreign exchange and commodities, since it keys off whichever instrument the ledger records.

Within each combination, approved settings are also filed by **market condition** — trending, ranging, or volatile — classified using established statistical measures rather than visual judgement. When conditions change, the engine proposes recalling the settings it previously learned for those conditions instead of relearning from scratch.

CONTINUITY

Evolution continues indefinitely while a bot runs, and **survives restarts**. Both the learned parameters and the full history are stored in the database and reapplied when bots resume. This was verified by restarting the application: bots returned running the evolved settings (take-profit 3.6%, stop-loss 1.6%) rather than reverting to the original values.

7. Defects found and fixed

Several of these were actively corrupting the figures shown on screen, which is why the underlying trading problem went unnoticed for as long as it did.

AREA	PROBLEM	RESOLUTION	
Trade counter	Counted rows in a database table that nothing ever writes to, so it permanently displayed zero while trades were being executed.	All counts, dates and resets repointed to the live trade ledger.	FIXED
Account reset	Called a database method that does not exist. Every reset reported success while deleting nothing.	Corrected; the deletion now commits first so a later failure cannot leave stale data behind.	FIXED
Report page	Crashed while loading because numeric database columns arrive as text and were being formatted as numbers.	Values converted before formatting; totals now read from the ledger.	FIXED
Bot persistence	Running bots existed only in memory and disappeared entirely whenever the application restarted.	Bot configurations are stored and automatically restored at startup.	FIXED
Chart timeframes	Two copies of the chart component existed; the one actually being served was outdated, so the 30-minute and 4-hour buttons silently did nothing.	Consolidated to a single served copy.	FIXED
Updates not appearing	No caching rules on stylesheets or scripts, so browsers continued serving old files and deployed changes appeared to have no effect.	Cache headers added along with asset versioning.	FIXED
Chart sizing	The chart rendered at 150 pixels tall inside a 424 pixel panel, leaving a large empty area beneath it.	Chart now fills its container; panel enlarged, with chart type and indicators made configurable.	FIXED
Mobile interface	The interface contained no responsive rules whatsoever — a fixed desktop layout was served to every device.	Full responsive layer added: stacked layout, drawer navigation, and no sideways scrolling.	FIXED

8. Infrastructure and access

- **Stable address.** The host machine's network address kept changing, repeatedly breaking the shared URL. It now holds a fixed address that survives reboots. The platform is reachable at `http://192.168.1.50:5050`.
- **Automatic startup.** The application and database restart automatically with the machine, and previously running bots resume by themselves. No manual step is required after a power cycle.
- **User account** created for Joel and verified against the live login.

NOTE ON THE REQUESTED ADDRESS

The specific address 192.168.1.7 could not be used: another device on the network currently holds it, and the operating system refuses to assign a duplicate address — doing so causes intermittent connection failures for both machines. Claiming that particular address requires a reservation on the router together with releasing it from the other device.

9. Outstanding items

ITEM	DETAIL	
Custom strategies cannot trade	Strategies created in the in-app builder are saved only in the browser. A bot started on one will run but never place a trade, because the server does not recognise the strategy. Making these execute server-side is a self-contained piece of work.	BLOCKER
Account balance resets on restart	Accumulated profit and loss is discarded from the equity figure each restart, so the dashboard understates cumulative performance. The ledger remains correct.	MISLEADING
Passwords stored in plain text	Login compares raw text rather than a hashed value. Acceptable for an internal prototype; must be resolved before any external exposure.	SECURITY

10. What success realistically looks like

It is worth being explicit, because expectations here determine whether the project is judged fairly. **No trading system wins every trade.** Markets contain genuine randomness, and any system or vendor claiming otherwise should be treated with suspicion.

The achievable goal is **positive expectancy**: over a large number of trades, the winners multiplied by their average size exceed the losers multiplied by theirs. A system can be profitable while losing more trades than it wins, provided the wins are larger.

THE DISTINCTION THAT MATTERS

The previous configuration was not merely unlucky — it was **arithmetically incapable** of profit, because its only profitable exit could not be reached. That is a solvable engineering problem, and it has been solved. The Evolution Engine makes the system progressively better calibrated to the market it is actually trading; it does not make it able to predict that market.

Recommended next steps

1. Allow the bots to run and accumulate completed trades. The first lesson becomes available after 8 closed trades, then every 5 thereafter.
2. When a strategy card on the home page shows **Lesson Ready**, open it, read the reasoning, and approve it if the argument holds.
3. Assess performance from the trade ledger rather than the dashboard equity figure until account persistence is addressed.
4. Decide whether server-side execution of custom strategies is the next priority.

11. Glossary

TERM	MEANING
Paper trading	Simulated trading against real live prices, using no real money.
Take-profit	The gain at which a position is automatically closed to bank a profit.
Stop-loss	The loss at which a position is automatically closed to limit damage.
Expectancy	Average profit or loss per trade. Positive expectancy means the system makes money over many trades.
Win rate	Percentage of trades that finish profitably. A high win rate does not guarantee profit if losses are large.
Risk-to-reward	Size of the intended gain relative to the accepted loss. A 1:3 ratio needs a 25% win rate to break even.
Whipsaw	Repeatedly entering and exiting as a signal alternates direction in a sideways market, accumulating small losses and fees.
Reversal	Closing a position and immediately opening the opposite one because the signal flipped.
Market regime	The prevailing character of the market — trending, ranging, or volatile. Different conditions demand different settings.
Trade ledger	The permanent record of every executed trade; the authoritative history of system activity.
Generation	One round of approved parameter changes produced by the Evolution Engine.

GoatBot Trade · Paper Trading V2 · Engineering Brief

All figures drawn from 26 trade ledger entries covering 5–10 August 2026. Every fix described has been deployed and verified against the running system.